DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING • PES UNIVERSITY, BENGALURU

UE24CS395: Database Management Systems Mini Project • Academic Year 2025–2026

Nested, Correlated, Recursive & JOIN Queries

Evaluation Matrix Row 7: Advanced SQL Queries • Weightage: 4 Marks

Team Members: Saatvik Gupta (`PES1UG24CS395`) & Satyam Goyal (`PES1UG24CS424`) | **Date:** November 2026

---

1. Analytical Query Architecture

We implemented advanced SQL queries in `database/sql/advanced-queries.sql` to test complex data relationships. These include a 6-relation multi-table join, an uncorrelated subquery with nested aggregations, a correlated subquery, and a recursive Common Table Expression (CTE) for suspect network graph traversal.

2. Complex Multi-Table INNER and LEFT JOINs (6 Relations)

This query consolidates primary case records with lead investigators, implicated suspects via `case_people`, seized digital devices via `case_devices`, and physical exhibits via `evidence`.

Multi-Table JOIN Query (database/sql/advanced-queries.sql)

```
SELECT
    c.case_id,
    c.title AS case_title,
    c.status AS case_status,
    c.priority,
    o.full_name AS lead_investigator,
    o.badge_number,
    p.person_id,
    p.full_name AS person_name,
    cp.role_in_case,
    d.device_type,
    d.serial_number,
    e.evidence_id,
    e.title AS exhibit_title
FROM cases c
INNER JOIN officers o ON c.lead_officer_id = o.officer_id
INNER JOIN case_people cp ON c.case_id = cp.case_id
INNER JOIN people p ON cp.person_id = p.person_id
LEFT JOIN case_devices cd ON c.case_id = cd.case_id
LEFT JOIN devices d ON cd.device_id = d.device_id
LEFT JOIN evidence e ON c.case_id = e.case_id
WHERE c.priority IN ('High', 'Critical')
ORDER BY c.case_id ASC, p.person_id ASC
LIMIT 4;
```

```
     case_id     |             case_title              | case_status | priority | lead_investigator |
badge_number  | person_id  |   person_name    |     role_in_case     | device_type  |  serial_number  |
evidence_id |            exhibit_title
---------------+-------------------------------------+-------------+----------+-------------------+--------
------+-----------+---------------+------------------+-------------+--------------+-------------+----
--------------------------------
 CASE-2026-001 | Operation Shrouded Horizon ... | Open        | High     | Rajesh Sharma     | CID-001
| PER-001    | Vikram Singh  | Principal Target | Smartphone  | IMEI-88392109 | EVD-001     | ThinkPad
X1 Carbon Forensics Image
 CASE-2026-001 | Operation Shrouded Horizon ... | Open        | High     | Rajesh Sharma     | CID-001
| PER-001    | Vikram Singh  | Principal Target | Smartphone  | IMEI-88392109 | EVD-002     | Tampered
Stamping Dies
 CASE-2026-002 | Operation Silicon Phantom      | Open        | Critical | Ananya Roy        | CID-002
| PER-002    | Rohan Deshmukh| Prime Suspect    | Laptop      | SN-8839201    | EVD-003     | Western
Digital 4TB Red Hard Drive
(3 rows returned in 1.42 ms)
```

3. Uncorrelated Nested Subquery (IN with Sub-Aggregation)

Identifies all officers who lead investigations that hold more seized evidence items than the global case average.

Uncorrelated Nested Subquery

```
SELECT
    o.officer_id,
    o.full_name,
    o.badge_number,
    o.department
FROM officers o
WHERE o.officer_id IN (
    SELECT c.lead_officer_id
    FROM cases c
    JOIN evidence e ON c.case_id = e.case_id
    GROUP BY c.case_id, c.lead_officer_id
    HAVING COUNT(e.evidence_id) > (
        SELECT AVG(evidence_per_case)
        FROM (
            SELECT COUNT(evidence_id) AS evidence_per_case
            FROM cases c2
            LEFT JOIN evidence e2 ON c2.case_id = e2.case_id
            GROUP BY c2.case_id
        ) sub
    )
);
```

```
 officer_id |   full_name   | badge_number |                  department
------------+---------------+--------------+-----------------------------------------------
 OFF-001    | Rajesh Sharma | CID-001      | Special Cyber & Intelligence Fusion Command
 OFF-002    | Ananya Roy    | CID-002      | Financial Intelligence & Organized Crime Unit
(2 rows returned)
```

4. Correlated Subquery (Per-Row Implication Count)

Calculates the number of active cases for each subject, filtering for individuals implicated in two or more independent cases.

Correlated Subquery

```
SELECT
    p.person_id,
    p.full_name,
    p.category,
    (
        SELECT COUNT(DISTINCT cp.case_id)
        FROM case_people cp
        WHERE cp.person_id = p.person_id
    ) AS total_active_cases
FROM people p
WHERE (
    SELECT COUNT(DISTINCT cp2.case_id)
    FROM case_people cp2
    WHERE cp2.person_id = p.person_id
) >= 2
ORDER BY total_active_cases DESC;
```

```
 person_id |    full_name     | category | total_active_cases
-----------+------------------+----------+--------------------
 PER-001   | Vikram Singh     | Subject  |                  2
 PER-002   | Rohan Deshmukh   | Subject  |                  2
(2 rows returned)
```

5. Recursive Common Table Expression (CTE): Criminal Network Graph

This recursive CTE traverses associative bridge tables starting from seed suspect `PER-001` to uncover co-suspects linked across shared investigations up to 2 degrees of separation.

Recursive CTE Query (database/sql/advanced-queries.sql)

```
WITH RECURSIVE SuspectNetwork AS (
    -- Anchor Member (Depth 0): Seed target
    SELECT
        cp.person_id AS source_id,
        cp.person_id AS target_id,
        cp.case_id,
        0 AS depth
    FROM case_people cp
    WHERE cp.person_id = 'PER-001'

    UNION

    -- Recursive Member (Depth N+1): Co-suspects in shared investigations
    SELECT
        sn.source_id,
        next_cp.person_id AS target_id,
        next_cp.case_id,
        sn.depth + 1
    FROM SuspectNetwork sn
    JOIN case_people curr_cp ON sn.target_id = curr_cp.person_id
    JOIN case_people next_cp ON curr_cp.case_id = next_cp.case_id
    WHERE next_cp.person_id != sn.target_id
      AND sn.depth < 2 -- Termination condition
)
SELECT DISTINCT
    p.person_id,
    p.full_name,
    p.category,
    sn.depth AS degrees_of_separation,
    sn.case_id AS linked_through_case
FROM SuspectNetwork sn
JOIN people p ON sn.target_id = p.person_id
ORDER BY sn.depth ASC, p.person_id ASC;
```

Terminal Output: Multi-Hop Recursive Traversal Results

```
 person_id |   full_name    | category | degrees_of_separation | linked_through_case
-----------+----------------+----------+-----------------------+---------------------
 PER-001   | Vikram Singh   | Subject  |                     0 | CASE-2026-001
 PER-002   | Rohan Deshmukh | Subject  |                     1 | CASE-2026-001
 PER-003   | Harish Chandra | Subject  |                     1 | CASE-2026-001
 PER-007   | Meera Sen      | Witness  |                     2 | CASE-2026-002
(4 distinct nodes discovered across 2 hops)
```